UGANDA CHRISTIAN UNIVERSITY

A Center of Excellence in the Heart of Africa

DEPARTMENT OF COMPUTING & TECHNOLOGY

Faculty of Engineering, Design & Technology

Group 18 Assignment Report

Naive Memoized Fibonacci

No.	Name	Registration Number
1	Ainebyona Sheba J.	M25B23/002
2	Nyakecho Karen Faith	M25B23/020
3	Isaac Isota	M25B23/043
4	Teta Nadine Tracy	S25B23/010
5	Precious Nimmusiima Abangira	M25B23/028

Course: Design and Analysis of Algorithms

Faculty: Faculty of Engineering, Design and Technology

Department: Department of Computing and Technology

Lecturer: Stephen Kalyesubula and John B. Wabwire Habere

Part 2: Experiments (Results & Graphs)

For this part of the assignment, we ran our Python implementations of both Algorithm A (the plain recursive version) and Algorithm B (the Memoized version). We recorded the execution times and the final counter values as n increased. Here is the data we collected:

n	Algorithm A (Counter)	Algorithm A Time (s)	Algorithm B (Counter)	Algorithm B Time (s)
5	15	0.000008	–	–
10	177	0.000069	19	0.000024
15	1973	0.000377	–	–
20	21,891	0.004146	–	–
25	242,785	0.047554	–	–
30	2,692,537	0.558391	–	–
35	29,860,703	5.734440	–	–
50	–	–	99	0.000053
100	–	–	199	0.000059
200	–	–	399	0.000200
500	–	–	999	0.000434
1000	–	–	Limit Hit	–

Graph Explanation (Log Scale)

When we plotted our results for n against the final counter values, we quickly realized that using a logarithmic scale on the y-axis was absolutely necessary for Algorithm A. Because the counter grows exponentially, a standard linear scale would squash the smaller values (like n=5 or 10) flat against the bottom of the graph, while the larger values would shoot straight up off the page. The log scale converts this massive exponential growth into a straight, readable line, making it much easier to actually see and compare the trends.

Part 3: Analysis

1. Recurrence Relations

Based on how the code is structured, we established the following recurrence relations for the counter:

Algorithm A (Plain Recursion): $T(n) = T(n-1) + T(n-2) + 1$, with our base cases being $T(0) = 1$ and $T(1) = 1$.

Algorithm B (Memoization): T(n) = 2n - 1 (assuming we start with a completely empty table).

2. Algorithm A Counter Growth & Complexity

To understand why Algorithm A gets so slow so fast, we can mathematically show that counter(n) = 2 × F(n+1) - 1.

Base Cases: For n=0, the formula gives 2(1) - 1 = 1 for the counter, which matches the code. For n=1, it also gives 2(1) - 1 = 1.

> Induction Step: If we assume this holds true for k < n, we can substitute it into our recurrence relation: counter(n) = counter(n-1) + counter(n-2) + 1 counter(n) = [2 × F(n) - 1] + [2 × F(n-1) - 1] + 1 counter(n) = 2(F(n) + F(n-1)) - 1 = 2 × F(n+1) - 1.

Because the Fibonacci sequence itself grows at a rate tied to the golden ratio (roughly 1.618), the counter grows at that same proportional rate. This perfectly explains the exponential running time we observed, which is usually written as an upper bound of O(2^n).

3. Algorithm B Counter Growth & Complexity

On the other hand, our data shows Algorithm B reaches exactly 2n - 1 on the counter when starting fresh. We noticed that for any new value k, fib(k) triggers exactly two recursive function calls. The first call to fib(k-1) pushes the recursion deeper, but the second call to fib(k-2) immediately finds its answer in our memoization table and returns instantly in O(1) time. This is why our data perfectly follows the 2n - 1 pattern (for example, n=50 resulted in a counter of exactly 99).

Time Complexity: O(n), or Linear time.

Space Complexity: O(n), also Linear. This accounts for the space needed for the dictionary/table and the maximum depth of the call stack.

4. Complexity Classes

Looking at our experimental data, Algorithm A is definitely in the Exponential class (O(2^n)). Every time we increased n by 5, the counter multiplied by about 11, which matches the math perfectly since 1.618^5 is about 11.09.

Algorithm B belongs to the Linear class (O(n)), as the counter scales in a direct, straight line with n.

5. Estimating n=1000 for Algorithm A

If we wanted to compute n=1000 with Algorithm A, the formula tells us it would result in a counter value of 2 × F(1001) - 1. Since F(1001) is a 208-digit number,

this is completely impossible. In our test, Python managed a counter of about 29.8 million in 5.73 seconds. At that speed, calculating n=1000 would take longer than the current age of the universe. Needless to say, it is completely unfeasible for this assignment.

Part 4: Pushing to n=1000

1. Default Settings Behaviour

When we first tried to run Algorithm B with n=1000, Python immediately crashed and threw a RecursionError. We looked this up and found that Python sets a default recursion limit (usually 1000) to prevent accidental infinite loops from eating up all system memory. Because Algorithm B has to recursively dig all the way down to n=1 before it can start returning and saving values, it hits this safety ceiling.

2. Proposed Fixes

To get around this limitation, we came up with two possible fixes:

Fix 1: Manually increase the limit. We can use sys.setrecursionlimit() to raise the ceiling. The trade-off is that while it is a very easy fix that does not require rewriting our code, it is a bit risky. If n gets too large, Python could run out of actual stack memory and crash the whole program.

Fix 2: Rewrite it iteratively. Instead of using top-down recursion, we can use a bottom-up loop (Dynamic Programming). This completely avoids the call stack, saving memory and eliminating recursion errors. The downside is that we had to rewrite the algorithm logic from scratch.

3. The if fibtable[n] Bug

While reviewing the provided pseudocode, we noticed a subtle quirk. If we write "if fibtable[n]:" directly in Python, it actually fails when n=0. Since F(0) is 0, and 0 evaluates to False in Python, the code assumes the value has not been cached and calculates it all over again. It does not break the final answer, but it is an interesting edge case that adds a tiny bit of unnecessary work.

Part 5: Conclusion

To conclude, as n approaches 1000, Algorithm B is clearly the superior choice by a massive margin. While Algorithm A chokes on its own exponential O(2^n) complexity and becomes literally impossible to run, Algorithm B breezes through the calculation in a fraction of a millisecond using linear O(n) time.

The magic behind this massive speedup is memoization, which takes advantage of overlapping subproblems. The naive recursive approach is incredibly wasteful

because it re-calculates the exact same branches of the Fibonacci tree millions of times. By simply caching the result of a subproblem the very first time we see it, we can instantly retrieve it later. This brilliant little trick collapses an impossibly huge exponential tree into a simple, straightforward linear path. **Source Code:**

The complete Python code for these implementations is available on GitHub. CLICK HERE to view the repository.